

Exercício - Capítulo 1, Engenharia de Software:

1 - O software em si não se trata apenas do programa em si, mas todos os passos desde a sua ideia inicial até a realização. Além do programa, podemos incluir sua documentação de dados, a documentação do sistema, a elaboração da estrutura do sistema, a documentação de configurações para a operação desse sistema.

2 - Os produtos genéricos são sistemas stand-alone produzidos para serem vendidos (ferramentas de bancos de dados, processadores de textos, pacotes gráficos, gerenciamento de projetos) ou aplicações verticais (sistemas de informação para bibliotecas, de contabilidade, de manutenção de registros). Já os sob demanda, o cliente encomenda um sistema específico (sistemas de controle eletrônicos, apoio ao processo de negócio, controle de tráfego). A principal diferença entre eles é que no genérico o desenvolvedor controla sua especificação, enquanto no sob demanda, quem controla sua especificação é a empresa cliente.

3 - O software deve ter manutenibilidade (deve evoluir para atender as demandas), confiabilidade (não deve causar prejuízos físicos ou econômicos), eficiência (boa capacidade de resposta, tempo de processamento e uso de memória), e aceitabilidade (ser compreensível, usável e compatível).

4 - Alguns problemas que podem surgir (e já são enfrentados no século 21) relacionados ao meio ambiente podem ser: consumo energético de data centers, a obsolescência programada indireta (onde a má otimização e sistemas mais pesados exigem computadores mais robustos e mais caros), produzir sistemas e arquiteturas que priorizem a eficiência energética e a redução de carbono. Além desses problemas ambientais, ainda podem surgir problemas como a manutenção de códigos de IA, excesso de programas antigos que necessitam ser atualizados.

5 - Por não seguir uma padrão na produção de softwares, a engenharia de softwares fornece alguns paradigmas importantes que ajudam a melhorar a produção dos mesmo, como por exemplo, analisar o fator de risco de usar aquele tipo de modelo, analisar se o ambiente e os recursos vão ser suficientes para a elaboração e execução, o tempo de vida de um software, entre outros.

6 - Algumas ideias da engenharia de software podem ser aplicadas a outros tipos de sistemas, como por exemplo, a abstração e modularidade, que consiste em dividir o problema em problemas menores, visando resolvê-los separadamente e então chegar a uma resolução completa do problema, ou por exemplo, a requisição de validação e garantia de qualidade, que visa manter a boa organização e o controle dos processos.

7 - Os serviços passaram a ficar disponíveis e altamente distribuídos. Além disso, os sistemas web fizeram com que as linguagens de programação avançassem exponencialmente e o reuso de software tornou-se mais comum entre os usuários.

8 - Apesar de não possuir um órgão fiscalizador de classe, como por exemplo a OAB para os advogados, ou o CRM para os médicos, existem algumas regras instituídas e adotadas pelos programadores de engenheiros de softwares, visando garantir o bom funcionamento da profissão e a qualidade dos produtos.

9 - Público: Um engenheiro de software trabalhando em uma montadora de veículos descobre que o código de controle do motor foi intencionalmente alterado para fraudar os testes de emissão de gases poluentes (como ocorreu no escândalo do *Dieseldgate*). Ele decide denunciar a prática, priorizando a saúde ambiental da sociedade sobre os lucros ilícitos da empresa.

Cliente e Empregador: Um desenvolvedor tem acesso a dados financeiros confidenciais de um banco para o qual presta serviço. Um amigo, que trabalha em uma empresa concorrente, pede "apenas algumas dicas" sobre como o sistema foi estruturado. O desenvolvedor se recusa a compartilhar qualquer detalhe, honrando o acordo de confidencialidade (NDA) e a lealdade ao cliente.

Produto: Uma equipe está desenvolvendo um software para controle de dosagem de medicamentos em hospitais. A diretoria pressiona para que o sistema seja lançado na sexta-feira para cumprir uma meta comercial, mas a equipe avisa que a suíte de testes de estresse não foi concluída. Os desenvolvedores se recusam a aprovar o lançamento, pois um bug em produção poderia custar vidas.

Julgamento: Uma engenheira sênior é encarregada de escolher um provedor de computação em nuvem para sua empresa. Uma das empresas concorrentes oferece a ela ingressos caros para um show internacional e viagens "de cortesia". Ela recusa os presentes para garantir que sua avaliação técnica permaneça 100% imparcial e baseada apenas no mérito tecnológico.

Gerência: Um gerente de projetos percebe que a equipe está atrasada para uma entrega. Em vez de impor semanas de horas extras abusivas e não remuneradas (conhecido como *crunch time*), ele renegocia o escopo do projeto com os clientes, protegendo a saúde mental de seus desenvolvedores e evitando que o esgotamento gere códigos de baixa qualidade.

Profissão: Um arquiteto de software descobre uma vulnerabilidade crítica (um erro *zero-day*) em um servidor web muito popular. Em vez de vender essa informação na *dark web* ou mantê-la em segredo para uso próprio, ele relata a falha de forma responsável aos criadores do servidor para que eles lancem um patch de segurança para toda a comunidade.

Colegas: Durante uma sessão de revisão de código (*Code Review*), um desenvolvedor sênior percebeu que um programador júnior cometeu um erro básico de lógica de matrizes. Em vez de ridicularizá-lo na frente da equipe ou aprovar o código com desprezo, o sênior senta com o júnior, explica didaticamente o erro e o orienta sobre as melhores práticas.

Pessoal: Um programador que passou os últimos 10 anos desenvolvendo sistemas internos se depara com a aprovação de novas leis de proteção de dados (como a LGPD). Por iniciativa própria, ele dedica horas de estudo para entender as novas exigências legais de criptografia e anonimização de dados, garantindo que suas habilidades não fiquem obsoletas ou perigosas para os futuros projetos.

10 - Alguns pontos importantes são levantados nessa discussão:

A primeira cláusula do Código de Ética da ACM/IEEE afirma que o engenheiro deve agir no interesse público. Prevenir um ataque terrorista que pode matar milhares de inocentes é o ato máximo de proteção ao interesse público. Ao mesmo tempo, quando os cidadãos sabem que estão sendo constantemente vigiados (efeito Panóptico), a liberdade de expressão, de associação e de pensamento é sufocada. Destruir a privacidade de toda uma nação destrói o próprio "público" que o sistema tentava proteger.

Outro ponto, é sobre o desvio de finalidade, onde um sistema construído hoje para rastrear células terroristas pode ser usado amanhã (por um governo mal-intencionado) para rastrear opositores políticos, jornalistas investigativos ou minorias.

Além disso, nenhum sistema é 100% seguro. Se houvesse um leak com essa quantidade colossal de dados das pessoas, teriam consequências catastróficas para a vida dessas pessoas.

O ideal é o engenheiro procurar analisar essas diversas situações antes de tentar desenvolver algo sem pensar. Levar em conta como o software vai afetar a vida das pessoas e o ambiente ao redor, eventuais medidas de segurança para lidar com alguns desses problemas futuramente, entre outros.